

Теоретический материал

Метод может возвращать значение, какой-либо результат для этого применяется оператор **return**, после которого идет возвращаемое значение:

return возвращаемое значение;

Методы с типом **void** не возвращают никакого значения. Они просто выполняют некоторые действия.

Наиболее простой способ передачи параметров представляет передача по значению, по сути это обычный способ передачи параметров:

void Increment(int n)

{

n++;

Console.WriteLine(\$"Число в методе Increment: {n}");

}

При передаче аргументов параметрам по значению параметр метода получает не саму переменную, а ее копию и далее работает с этой копией независимо от самой переменной.

Increment(number);

При передаче параметров по ссылке перед параметрами используется модификатор **ref**:

```

void Increment(ref int n)

{

    n++;

    Console.WriteLine($"Число в методе Increment: {n}");

}

```

При передаче значений параметрам по ссылке метод получает адрес переменной в памяти. И, таким образом, если в методе изменяется значение параметра, передаваемого по ссылке, то также изменяется и значение переменной.

Обратите внимание, что модификатор **ref** указывается как перед параметром при объявлении метода, так и при вызове метода перед аргументом, который передается параметру.

```

Increment(ref number);

```

Параметры могут быть также выходными. Чтобы сделать параметр выходным, перед ним ставится модификатор **out**:

```

void Sum(int x, int y, out int result)

{

    result = x + y;

}

```

Причем, как и в случае с **ref** ключевое слово **out** используется как при определении метода, так и при его вызове.

Sum(10, 15, out number);

Кроме выходных параметров с модификатором **out** метод может использовать входные параметры с модификатором **in**.

Модификатор **in** указывает, что данный параметр будет передаваться в метод по ссылке, однако внутри метода его значение параметра нельзя будет изменить. Например, возьмем следующий метод:

```
void GetRectangleData(in int width, in int height, out int rectArea,  
out int rectPerimetr)
```

```
{
```

```
    //width = 25; // нельзя изменить, так как width - входной  
    параметр
```

```
    rectArea = width * height;
```

```
    rectPerimetr = (width + height) * 2;
```

```
}
```

Вызов:

```
GetRectangleData(w, h, out var area, out var perimetr); // in w, in h
```

В примерах выше можно было изменять значение **ref**-параметра. Однако иногда это может быть нежелательно. И чтобы гарантировать, что **ref**-параметр не изменит своего значения, начиная с версии C# 12 можно применять **ref**-параметры только для чтения. Такие параметры предваряются ключевым словом **readonly**:

```
void Increment(ref readonly int n)
```

```
{
```

```
// n++; // нельзя, иначе будет ошибка компиляции  
  
Console.WriteLine($"Число в методе Increment: {n}");  
  
}
```

Вызов:

```
Increment(ref number);
```

Во всех предыдущих примерах мы использовали постоянное число параметров. Но, используя ключевое слово **params**, мы можем передавать неопределенное количество параметров:

```
void Sum(params int[] numbers)  
  
{  
  
    int result = 0;  
  
    foreach (var n in numbers)  
  
    {  
  
        result += n;  
  
    }  
  
    Console.WriteLine(result);  
  
}
```

Сам параметр с ключевым словом **params** при определении метода должен представлять одномерный массив того типа, данные которого мы собираемся использовать.

При вызове метода на место параметра с модификатором **params** мы можем передать как отдельные значения, так и массив значений, либо

вообще не передавать параметры. Количество передаваемых значений в метод неопределённо, однако все эти значения должны соответствовать типу параметра с **params**.

Вызов:

```
int[] nums = { 1, 2, 3, 4, 5};
```

```
Sum(nums);
```

```
Sum(1, 2, 3, 4);
```

```
Sum(1, 2, 3);
```

```
Sum();
```

Задание 1

Задача:

Система заказов

Разработать ПО со следующей архитектурой классов и функционалом:

Класс «Блюдо»:

Хранимая информация:

- Id блюда (целое число);
- Название (тип строка);
- Состав (тип строка);
- Вес (строка формата (100/20/50));
- Цена (вещественный тип);
- Категория (Тип перечисление (напитки, салаты, холодные закуски, горячие закуски, супы, горячие блюда, десерт и т.д));
- Время готовки (целое число);
- Тип (массив строк (острое, веганское, халяль, кошерное и т. д.).

Методы:

- Создание блюда;
- Редактирование блюда;
- Вывод информации о блюде;
- Удаление блюда.

Класс «Заказ»:

Хранимая информация:

- Id заказа (целое число);
- Id стола (целое число);

- Массив блюд (объекты класса, учесть возможность дублирования блюда);
- Комментарий (строка);
- Время принятия заказа (тип время или строка);
- Официант (целое число);
- Время закрытия заказа (тип время или строка);
- Итоговая стоимость (Вещественное число).

Методы:

- Создание заказа;
- Изменение заказа;
- Вывод информации о заказе;
- Закрытие заказа;
- Вывод чека (только для закрытых заказов).

Столик:

Официант:

Период обслуживания: с по....

Категория блюда_1:

Название блюда_1	кол-во*цена=итог цена
Название блюда_2	кол-во*цена=итог цена
Название блюда_3	кол-во*цена=итог цена
	Под_итог категории

...

Категория блюда_n:

Название блюда_1	кол-во*цена=итог цена
Название блюда_2	кол-во*цена=итог цена
Название блюда_3	кол-во*цена=итог цена
	Под_итог категории

Итог счета: итог цена

Общие требования к функционалу:

- Программный продукт должен позволять создавать набор из n ($n > 0$) блюд (каждое блюдо представляет собой объект класса);
- Программный продукт должен позволять создавать набор из n ($n > 0$) заказов (каждый заказ представляет собой объект класса);
- Метод вывода меню (блюда должны быть распределены по категориям и результат работы метода должен содержать значимую для клиента информацию, в дальнейшем планируется печать меню);
- Метод подсчета стоимости всех закрытых заказов на текущий момент;
- Метод подсчета закрытых заказов конкретного официанта на текущий момент;
- Метод сбора статистики по количеству заказанных блюд.

Итоговый проект должен содержать 3 файла классов.

Использовать возможности: in, out, ref, params.

Решение:

```
namespace restaurant
{
    public enum Category
    {
        Soup,
        Salad,
        MainCourse,
        Dessert,
        Beverage,
        Appetizer
    }

    public class Dish
    {
        public readonly int Id;
        public string Name { get; private set; }
        public string Composition { get; private set; }
        public string Weight { get; private set; }
        public double Price { get; private set; }
        public Category Category { get; private set; }
        public int CookingTime { get; private set; }
        public string[] Type { get; private set; }

        private static int _nextId = 1;

        public Dish(string name, string composition, string weight, double price, Category category, int cookingTime, string[] type)
        {
            Id = _nextId++;
            Name = name;
            Composition = composition;
            Weight = weight;
            Price = price;
        }
    }
}
```



```

        Category = category;
        CookingTime = cookingTime;
        Type = type ?? Array.Empty<string>();
    }

    public static bool ValidateWeightFormat(string weight)
    {
        if (string.IsNullOrEmpty(weight))
            return false;

        var parts = weight.Split('/');
        foreach (var part in parts)
        {
            if (!int.TryParse(part, out int value) || value <= 0)
                return false;
        }
        return true;
    }

    public void EditDish(
        bool changeName = false, string? newName = null,
        bool changeComposition = false, string? newComposition = null,
        bool changeWeight = false, string? newWeight = null,
        bool changePrice = false, double newPrice = 0,
        bool changeCategory = false, Category newCategory = default,
        bool changeCookingTime = false, int newCookingTime = 0,
        bool changeType = false, string[]? newType = null)
    {
        if (changeName && !string.IsNullOrEmpty(newName))
            Name = newName;

        if (changeComposition && !string.IsNullOrEmpty(newComposition))
            Composition = newComposition;

        if (changeWeight && ValidateWeightFormat(newWeight ?? ""))
            Weight = newWeight;

        if (changePrice && newPrice > 0)
            Price = newPrice;

        if (changeCategory)
            Category = newCategory;

        if (changeCookingTime && newCookingTime > 0)
            CookingTime = newCookingTime;

        if (changeType && newType != null)
            Type = newType;
    }

    public void PrintDishInfo()
    {
        Console.WriteLine($"ID: {Id}");
        Console.WriteLine($"Название: {Name}");
        Console.WriteLine($"Состав: {Composition}");
        Console.WriteLine($"Вес: {Weight}");
        Console.WriteLine($"Цена: {Price:F2} руб.");
        Console.WriteLine($"Категория: {Category}");
        Console.WriteLine($"Время готовки: {CookingTime} мин.");
        Console.WriteLine($"Типы: {string.Join(", ", Type)}");
        Console.WriteLine("-----");
    }
}

using System;
using System.Collections.Generic;
using System.Linq;

namespace restaurant
{
    public class Order(int tableId, string comment, int waiterId)
    {
        public readonly int OrderId = _nextOrderId++;
        public int TableId { get; set; } = tableId;
        public string Comment { get; set; } = comment;
        public int WaiterId { get; set; } = waiterId;
        public DateTime OrderTime { get; } = DateTime.Now;
        public DateTime? CloseTime { get; private set; }
    }
}

```

```

public bool IsClosed { get; private set; } = false;
public double TotalCost { get; private set; } = 0;

private readonly List<Dish> _dishes = [];

private static int _nextOrderId = 1;

public IReadOnlyList<Dish> Dishes => _dishes.AsReadOnly();

public void AddDish(Dish dish)
{
    if (IsClosed)
        throw new InvalidOperationException("Нельзя добавлять блюда в закрытый заказ.");

    _dishes.Add(dish);
    TotalCost += dish.Price;
}

public void ChangeOrder(
    bool changeTableId = false, int newTableId = 0,
    bool changeComment = false, string newComment = "",
    bool changeWaiterId = false, int newWaiterId = 0)
{
    if (IsClosed)
        throw new InvalidOperationException("Нельзя изменять закрытый заказ.");

    if (changeTableId && newTableId > 0)
        TableId = newTableId;

    if (changeComment)
        Comment = newComment;

    if (changeWaiterId && newWaiterId > 0)
        WaiterId = newWaiterId;
}

public void CloseOrder()
{
    if (IsClosed)
        return;

    IsClosed = true;
    CloseTime = DateTime.Now;
}

public void PrintOrderInfo()
{
    Console.WriteLine($"Заказ ID: {OrderId}");
    Console.WriteLine($"Столик: {TableId}");
    Console.WriteLine($"Официант ID: {WaiterId}");
    Console.WriteLine($"Комментарий: {Comment}");
    Console.WriteLine($"Время создания: {OrderTime:yyyy-MM-dd HH:mm:ss}");
    Console.WriteLine($"Статус: {(IsClosed ? $"Закрыт в {CloseTime:HH:mm:ss}" : "Открыт")}");
    Console.WriteLine($"Общая стоимость: {TotalCost:F2} руб.");

    Console.WriteLine("\nБлюда в заказе:");
    if (_dishes.Count == 0)
    {
        Console.WriteLine("Нет блюд");
    }
    else
    {
        foreach (var dish in _dishes)
        {
            Console.WriteLine($"- {dish.Name} ({dish.Price:F2} руб.)");
        }
    }
    Console.WriteLine("-----");
}

public void PrintCheck()
{
    if (!IsClosed)
    {
        Console.WriteLine("Чек можно распечатать только для закрытого заказа.");
        return;
    }

    Console.WriteLine("*****");

```

```

        Console.WriteLine($"Столик: {TableId}");
        Console.WriteLine($"Официант: {WaiterId}");
        Console.WriteLine($"Период обслуживания: с {OrderTime:HH:mm:ss} по {CloseTime:HH:mm:ss}");
        Console.WriteLine();

        // Группируем блюда по категориям
        var groupedDishes = _dishes.GroupBy(d => d.Category);
        double grandTotal = 0;

        foreach (var categoryGroup in groupedDishes)
        {
            Console.WriteLine($"{categoryGroup.Key}:");

            // Группируем блюда внутри категории
            var dishCounts = categoryGroup.GroupBy(d => d.Name)
                .Select(g => new {
                    Name = g.Key,
                    Dish = g.First(),
                    Count = g.Count(),
                    SubTotal = g.First().Price * g.Count()
                }).ToList();

            foreach (var dishCount in dishCounts)
            {
                Console.WriteLine($" {dishCount.Name} {dishCount.Count} * {dishCount.Dish.Price:F2} = {dishCount.SubTotal:F2}");
            }

            double categoryTotal = dishCounts.Sum(dc => dc.SubTotal);
            Console.WriteLine($" Под_итог категории: {categoryTotal:F2}");
            Console.WriteLine();

            grandTotal += categoryTotal;
        }

        Console.WriteLine($"Итого счета: {grandTotal:F2}");
        Console.WriteLine("*****");
    }
}

using System.Reflection.PortableExecutable;

namespace restaurant
{
    public class OrderSystemManager
    {
        public double TotalCost = 0;
        public List<Dish> Dishes = new List<Dish>();
        public List<Order> Orders = new List<Order>();

        public void PrintMenu()
        {
            foreach (var dish in this.Dishes)
            {
                dish.PrintDishInfo();
            }
        }

        public bool DeleteDish(int dishId)
        {
            var dishToRemove = Dishes.FirstOrDefault(d => d.Id == dishId);
            if (dishToRemove != null)
            {
                Dishes.Remove(dishToRemove);
                TotalCost -= dishToRemove.Price;
                return true;
            }
            return false;
        }

        public Order CreateOrder(int tableId, int waiterId, string comment)
        {
            Order order = new Order(tableId, comment, waiterId);
            this.Orders.Add(order);
            return order;
        }

        public int GetWaiterClosedOrdersCount(int waiterId)
        {

```

```

        return Orders.Count(order => order.WaiterId == waiterId && order.IsClosed);
    }

    public double CalculateTotalRevenue()
    {
        return Orders.Where(o => o.IsClosed).Sum(o => o.TotalCost);
    }

    public bool ValidateDishInput(string name, string composition, string weight, double price, Category category, int cookingTime,
string[] types, out string message)
    {
        message = "";

        if (string.IsNullOrEmpty(name))
        {
            message = "Название блюда не может быть пустым.";
            return false;
        }

        if (string.IsNullOrEmpty(composition))
        {
            message = "Состав блюда не может быть пустым.";
            return false;
        }

        if (!Dish.ValidateWeightFormat(weight))
        {
            message = "Неверный формат веса. Ожидается формат: 100/20/50 (числа, разделенные слэшами).";
            return false;
        }

        if (price <= 0)
        {
            message = "Цена должна быть положительным числом.";
            return false;
        }

        if (cookingTime <= 0)
        {
            message = "Время готовки должно быть положительным числом.";
            return false;
        }

        return true;
    }

    public Dish? GetDishById(int id)
    {
        return Dishes.FirstOrDefault(d => d.Id == id);
    }

    public Order? GetOrderById(int id)
    {
        return Orders.FirstOrDefault(o => o.OrderId == id);
    }

    public void AddDish(params Dish[] dishes)
    {
        foreach (var dish in dishes)
        {
            Dishes.Add(dish);
            TotalCost += dish.Price;
        }
    }

    public void PrintDishStatistics()
    {
        Console.WriteLine("\n=== Статистика по проданным блюдам ===");

        // Получаем все закрытые заказы
        var closedOrders = Orders.Where(o => o.IsClosed).ToList();

        if (closedOrders.Count == 0)
        {
            Console.WriteLine("Нет закрытых заказов для анализа.");
            return;
        }

        // Собираем все проданные блюда из закрытых заказов

```

```

var allSoldDishes = closedOrders
    .SelectMany(o => o.Dishes) // Нужно добавить этот метод в класс Order
    .ToList();

if (allSoldDishes.Count == 0)
{
    Console.WriteLine("Нет проданных блюд.");
    return;
}

// Группируем проданные блюда по названию
var dishStats = allSoldDishes
    .GroupBy(d => d.Name)
    .Select(g => new {
        Name = g.Key,
        TotalCount = g.Count(), // Количество продаж этого блюда
        TotalRevenue = g.Sum(d => d.Price), // Общая выручка от этого блюда
        AveragePrice = g.Average(d => d.Price), // Средняя цена (обычно равна цене блюда)
        Category = g.First().Category // Категория блюда
    })
    .OrderByDescending(s => s.TotalRevenue)
    .ToList();

if (dishStats.Count == 0)
{
    Console.WriteLine("Нет данных для статистики.");
    return;
}

var header = $"{"Блюдо",-15} | {"Кол-во",6} | {"Выручка",10} | {"Сред. цена",10} | {"Категория",12}";
Console.WriteLine(header);
Console.WriteLine(new string('-', header.Length));

foreach (var stat in dishStats)
{
    Console.WriteLine($"{"stat.Name",-15} | {"stat.TotalCount,6} | {"stat.TotalRevenue,10:F2} | {"stat.AveragePrice,10:F2} | {"stat.Category,12}");
}

Console.WriteLine("\nОбщая статистика:");
Console.WriteLine($"Всего продано блюд: {allSoldDishes.Count}");
Console.WriteLine($"Уникальных блюд продано: {dishStats.Count}");
Console.WriteLine($"Общая выручка от продаж: {allSoldDishes.Sum(d => d.Price):F2} руб.");

// Топ-3 самых продаваемых блюд
var top3 = dishStats.Take(3).ToList();
Console.WriteLine("\nТоп-3 самых продаваемых блюд:");
for (int i = 0; i < Math.Min(3, top3.Count); i++)
{
    Console.WriteLine($"{i + 1}. {top3[i].Name} - {top3[i].TotalCount} продаж, выручка: {top3[i].TotalRevenue:F2} руб.");
}
}
}

using restaurant;

class Program
{
    public static OrderSystemManager manager = new ();
    public static Random random = new ();

    static void Main()
    {
        // Добавляем блюда с полной информацией (включая обязательные поля)
        manager.AddDish([
            new ("Борщ", "Свекла, капуста, картофель, мясо, вода", "300/50/30", 150.0, Category.Soup, 20, ["горячее"]),
            new ("Оливье", "Картофель, морковь, огурцы, колбаса", "200/30", 120.0, Category.Salad, 10, ["холодное"]),
            new ("Компот", "Ягоды, вода", "250", 80.0, Category.Beverage, 5, ["веганское"])
        ]);

        bool running = true;
        while (running)
        {
            ShowMainMenu();

            var input = Console.ReadLine();
            if (string.IsNullOrEmpty(input) || !byte.TryParse(input, out byte choice))
            {

```

```

        Console.Clear();
        continue;
    }

    running = Act(choice, running);
}

static void ViewAllOrders()
{
    if (manager.Orders.Count == 0)
    {
        Console.WriteLine("Нет созданных заказов.");
        return;
    }

    Console.WriteLine("\nСписок всех заказов:");
    Console.WriteLine($"{ "ID",-4} | { "Стол",-4} | { "Оф.",-4} | { "Статус",-8} | { "Сумма",-8} | { "Время",-10}");
    Console.WriteLine(new string('-', 50));

    foreach (var order in manager.Orders)
    {
        string status = order.IsClosed ? "Закрыт" : "Открыт";
        Console.WriteLine($"{ order.OrderId,-4} | { order.TableId,-4} | { order.WaiterId,-4} | { status,-8} | { order.TotalCost,-8:F2} | {order.OrderTime:HH:mm:ss,-10}");
    }
}

static bool Act(int choice, bool running)
{
    switch (choice)
    {
        case 1:
            AddDish();
            break;
        case 2:
            EditDish();
            break;
        case 3:
            DeleteDish();
            break;
        case 4:
            ViewDish();
            break;
        case 5:
            AddOrder();
            break;
        case 6:
            EditOrder();
            break;
        case 7:
            CloseOrder();
            break;
        case 8:
            ViewOrder();
            break;
        case 9:
            PrintOrderCheck();
            break;
        case 10:
            manager.PrintMenu();
            break;
        case 11:
            ViewTotalRevenue();
            break;
        case 12:
            ViewWaiterStats();
            break;
        case 13:
            PrintDishStatistics();
            break;
        case 14:
            ViewAllOrders();
            break;
        case 15:
            ViewAllDishes();
            break;
        case 0:
    }
}

```

```

        Console.WriteLine("Выход из программы...");
        return false;
    default:
        Console.WriteLine("Неверный выбор. Попробуйте снова.");
        return true;
    }
}

if (running && choice != 0)
{
    Console.WriteLine("\nНажмите любую клавишу для продолжения...");
    Console.ReadKey();
    Console.Clear();
    return true;
}
return true;
}

static void ViewAllDishes()
{
    Console.WriteLine("\n=== Все блюда в меню ===");
    manager.PrintMenu();
}

static void ShowMainMenu()
{
    string menu = "Выберите действие (положительное целое число, либо ноль)\n\n" +
        "=== БЛЮДА ===\n" +
        "1. Добавить блюдо по ID\n" +
        "2. Редактировать блюдо по ID\n" +
        "3. Удалить блюдо по ID\n" +
        "4. Просмотреть блюдо по ID\n\n" +
        "=== ЗАКАЗЫ ===\n" +
        "5. Создать заказ\n" +
        "6. Изменить заказ по ID\n" +
        "7. Закрыть заказ по ID\n" +
        "8. Просмотреть заказ по ID\n" +
        "9. Распечатать чек заказа по ID\n\n" +
        "=== ОБЩЕЕ ===\n" +
        "10. Просмотреть меню\n" +
        "11. Общая выручка\n" +
        "12. Статистика официанта\n" +
        "13. Статистика по блюдам\n" +
        "14. Просмотреть все заказы\n" +
        "15. Просмотреть все блюда\n" +
        "0. Выход\n";

    Console.WriteLine(menu);
}

static int GetIntInput(string prompt = "Введите число: ")
{
    int result;
    while (true)
    {
        Console.Write(prompt);
        if (int.TryParse(Console.ReadLine(), out result))
        {
            return result;
        }
        Console.WriteLine("Ошибка ввода. Попробуйте еще раз.");
    }
}

static double GetDoubleInput(string prompt = "Введите число: ")
{
    double result;
    while (true)
    {
        Console.Write(prompt);
        if (double.TryParse(Console.ReadLine(), out result))
        {
            return result;
        }
        Console.WriteLine("Ошибка ввода. Попробуйте еще раз.");
    }
}

static void AddDish()
{

```

```

string name;
do
{
    Console.WriteLine("Введите название блюда (не может быть пустым): ");
    name = Console.ReadLine() ?? "";
    if (string.IsNullOrEmpty(name))
        Console.WriteLine("Название блюда не может быть пустым.");
} while (string.IsNullOrEmpty(name));

string composition;
do
{
    Console.WriteLine("Введите состав блюда (не может быть пустым): ");
    composition = Console.ReadLine() ?? "";
    if (string.IsNullOrEmpty(composition))
        Console.WriteLine("Состав блюда не может быть пустым.");
} while (string.IsNullOrEmpty(composition));

string weight;
do
{
    Console.WriteLine("Введите вес блюда (формат: 100/20/50): ");
    weight = Console.ReadLine() ?? "";
    if (!Dish.ValidateWeightFormat(weight))
        Console.WriteLine("Неверный формат веса. Ожидается формат: 100/20/50 (числа, разделенные слэшами).");
} while (!Dish.ValidateWeightFormat(weight));

double price;
do
{
    Console.WriteLine("Введите цену блюда (положительное число): ");
    price = GetDoubleInput();
    if (price <= 0)
        Console.WriteLine("Цена должна быть положительным числом.");
} while (price <= 0);

Console.WriteLine("Выберите категорию:");
var categories = Enum.GetValues<Category>();
for (int i = 0; i < categories.Length; i++)
{
    Console.WriteLine($"{i}. {categories.GetValue(i)}");
}

int catIndex;
do
{
    Console.WriteLine("Введите номер категории: ");
    catIndex = GetIntInput();
    if (catIndex < 0 || catIndex >= categories.Length)
        Console.WriteLine($"Номер должен быть от 0 до {categories.Length - 1}");
} while (catIndex < 0 || catIndex >= categories.Length);

Category category = (Category)categories.GetValue(catIndex);

int cookingTime;
do
{
    Console.WriteLine("Введите время готовки (в минутах, положительное число): ");
    cookingTime = GetIntInput();
    if (cookingTime <= 0)
        Console.WriteLine("Время готовки должно быть положительным числом.");
} while (cookingTime <= 0);

Console.WriteLine("Введите типы блюда через запятую (например, острое, веганское): ");
string typesInput = Console.ReadLine() ?? "";
string[] types = typesInput.Split(',', StringSplitOptions.RemoveEmptyEntries)
    .Select(t => t.Trim())
    .ToArray();

// Проверка валидности ввода
if (manager.ValidateDishInput(name, composition, weight, price, category, cookingTime, types, out string message))
{
    Dish dish = new Dish(name, composition, weight, price, category, cookingTime, types);
    manager.AddDish(dish);
    Console.WriteLine($"Блюдо добавлено успешно! Назначен ID: {dish.Id}");
}
else
{
    Console.WriteLine($"Ошибка добавления блюда: {message}");
}

```



```

    }
}

static void EditDish()
{
    Console.WriteLine("Введите ID блюда для редактирования: ");
    int id = GetIntInput();
    ViewAllDishes();

    var dish = manager.GetDishById(id);
    if (dish == null)
    {
        Console.WriteLine("Блюдо с таким ID не найдено.");
        return;
    }

    dish.PrintDishInfo();

    Console.WriteLine("Что вы хотите изменить?");
    Console.WriteLine("1. Название");
    Console.WriteLine("2. Состав");
    Console.WriteLine("3. Вес");
    Console.WriteLine("4. Цена");
    Console.WriteLine("5. Категория");
    Console.WriteLine("6. Время готовки");
    Console.WriteLine("7. Типы");
    Console.WriteLine("8. Всё сразу");
    Console.WriteLine("Выберите: ");

    int choice = GetIntInput();

    try
    {
        switch (choice)
        {
            case 1:
                string newName;
                do
                {
                    Console.WriteLine("Введите новое название (не может быть пустым): ");
                    newName = Console.ReadLine() ?? "";
                    if (string.IsNullOrEmpty(newName))
                        Console.WriteLine("Название блюда не может быть пустым.");
                } while (string.IsNullOrEmpty(newName));
                dish.EditDish(changeName: true, newName: newName);
                break;
            case 2:
                string newComposition;
                do
                {
                    Console.WriteLine("Введите новый состав (не может быть пустым): ");
                    newComposition = Console.ReadLine() ?? "";
                    if (string.IsNullOrEmpty(newComposition))
                        Console.WriteLine("Состав блюда не может быть пустым.");
                } while (string.IsNullOrEmpty(newComposition));
                dish.EditDish(changeComposition: true, newComposition: newComposition);
                break;
            case 3:
                string newWeight;
                do
                {
                    Console.WriteLine("Введите новый вес (формат: 100/20/50): ");
                    newWeight = Console.ReadLine() ?? "";
                    if (!Dish.ValidateWeightFormat(newWeight))
                        Console.WriteLine("Неверный формат веса. Ожидается формат: 100/20/50 (числа, разделенные слэшами).");
                } while (!Dish.ValidateWeightFormat(newWeight));
                dish.EditDish(changeWeight: true, newWeight: newWeight);
                break;
            case 4:
                double newPrice;
                do
                {
                    Console.WriteLine("Введите новую цену (положительное число): ");
                    newPrice = GetDoubleInput();
                    if (newPrice <= 0)
                        Console.WriteLine("Цена должна быть положительным числом.");
                } while (newPrice <= 0);
                dish.EditDish(changePrice: true, newPrice: newPrice);
                break;
        }
    }
}

```

```

case 5:
    Console.WriteLine("Выберите новую категорию:");
    var categories = Enum.GetValues(typeof(Category));
    for (int i = 0; i < categories.Length; i++)
    {
        Console.WriteLine($"{i}. {categories.GetValue(i)}");
    }
    int catIndex;
    do
    {
        Console.Write("Введите номер категории: ");
        catIndex = GetIntInput();
        if (catIndex < 0 || catIndex >= categories.Length)
            Console.WriteLine($"Номер должен быть от 0 до {categories.Length - 1}");
    } while (catIndex < 0 || catIndex >= categories.Length);
    Category newCategory = (Category)categories.GetValue(catIndex);
    dish.EditDish(changeCategory: true, newCategory: newCategory);
    break;
case 6:
    int newCookingTime;
    do
    {
        Console.Write("Введите новое время готовки (положительное число): ");
        newCookingTime = GetIntInput();
        if (newCookingTime <= 0)
            Console.WriteLine("Время готовки должно быть положительным числом.");
    } while (newCookingTime <= 0);
    dish.EditDish(changeCookingTime: true, newCookingTime: newCookingTime);
    break;
case 7:
    Console.Write("Введите новые типы через запятую: ");
    string typesInput = Console.ReadLine() ?? "";
    string[] newTypes = typesInput.Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(t => t.Trim())
        .ToArray();
    dish.EditDish(changeType: true, newType: newTypes);
    break;
case 8:
    // Редактирование всех полей с проверками
    string allName;
    do
    {
        Console.Write("Введите новое название (не может быть пустым): ");
        allName = Console.ReadLine() ?? "";
        if (string.IsNullOrEmpty(allName))
            Console.WriteLine("Название блюда не может быть пустым.");
    } while (string.IsNullOrEmpty(allName));

    string allComposition;
    do
    {
        Console.Write("Введите новый состав (не может быть пустым): ");
        allComposition = Console.ReadLine() ?? "";
        if (string.IsNullOrEmpty(allComposition))
            Console.WriteLine("Состав блюда не может быть пустым.");
    } while (string.IsNullOrEmpty(allComposition));

    string allWeight;
    do
    {
        Console.Write("Введите новый вес (формат: 100/20/50): ");
        allWeight = Console.ReadLine() ?? "";
        if (!Dish.ValidateWeightFormat(allWeight))
            Console.WriteLine("Неверный формат веса. Ожидается формат: 100/20/50 (числа, разделенные слэшами).");
    } while (!Dish.ValidateWeightFormat(allWeight));

    double allPrice;
    do
    {
        Console.Write("Введите новую цену (положительное число): ");
        allPrice = GetDoubleInput();
        if (allPrice <= 0)
            Console.WriteLine("Цена должна быть положительным числом.");
    } while (allPrice <= 0);

    Console.WriteLine("Выберите новую категорию:");
    categories = Enum.GetValues(typeof(Category));
    for (int i = 0; i < categories.Length; i++)
    {

```

```

        Console.WriteLine($"{i}. {categories.GetValue(i)}");
    }
    int allCatIndex;
    do
    {
        Console.Write("Введите номер категории: ");
        allCatIndex = GetIntInput();
        if (allCatIndex < 0 || allCatIndex >= categories.Length)
            Console.WriteLine($"Номер должен быть от 0 до {categories.Length - 1}");
    } while (allCatIndex < 0 || allCatIndex >= categories.Length);
    Category allCategory = (Category)categories.GetValue(allCatIndex);

    int allCookingTime;
    do
    {
        Console.Write("Введите новое время готовки (положительное число): ");
        allCookingTime = GetIntInput();
        if (allCookingTime <= 0)
            Console.WriteLine("Время готовки должно быть положительным числом.");
    } while (allCookingTime <= 0);

    Console.Write("Введите новые типы через запятую: ");
    string allTypesInput = Console.ReadLine() ?? "";
    string[] allTypes = allTypesInput.Split(',', StringSplitOptions.RemoveEmptyEntries)
        .Select(t => t.Trim())
        .ToArray();

    dish.EditDish(
        changeName: true, newName: allName,
        changeComposition: true, newComposition: allComposition,
        changeWeight: true, newWeight: allWeight,
        changePrice: true, newPrice: allPrice,
        changeCategory: true, newCategory: allCategory,
        changeCookingTime: true, newCookingTime: allCookingTime,
        changeType: true, newType: allTypes
    );
    break;
default:
    Console.WriteLine("Неверный выбор.");
    return;
}

Console.WriteLine("Блюдо успешно отредактировано!");
}
catch (ArgumentException ex)
{
    Console.WriteLine($"Ошибка при редактировании: {ex.Message}");
}
}

static void DeleteDish()
{
    Console.Write("Введите ID блюда для удаления: ");
    int id = GetIntInput();
    ViewAllDishes();

    var dish = manager.GetDishById(id);
    if (dish == null)
    {
        Console.WriteLine("Блюдо с таким ID не найдено.");
        return;
    }

    dish.PrintDishInfo();

    Console.Write("Вы уверены, что хотите удалить это блюдо? (y/n): ");
    var confirm = Console.ReadLine() ?? "";

    if (confirm.Equals("y", StringComparison.OrdinalIgnoreCase) || confirm.Equals("yes", StringComparison.OrdinalIgnoreCase))
    {
        if (manager.DeleteDish(id))
        {
            Console.WriteLine("Блюдо удалено успешно!");
        }
        else
        {
            Console.WriteLine("Ошибка при удалении блюда.");
        }
    }
}

```

```

    else
    {
        Console.WriteLine("Удаление отменено.");
    }
}

static void ViewDish()
{
    Console.Write("Введите ID блюда для просмотра: ");
    int id = GetIntInput();

    var dish = manager.GetDishById(id);
    if (dish == null)
    {
        Console.WriteLine("Блюдо с таким ID не найдено.");
        return;
    }

    dish.PrintDishInfo();
}

static void AddOrder()
{
    int tableId;
    do
    {
        Console.Write("Введите ID стола (положительное число): ");
        tableId = GetIntInput();
        if (tableId <= 0)
            Console.WriteLine("ID стола должно быть положительным числом.");
    } while (tableId <= 0);

    int waiterId;
    do
    {
        Console.Write("Введите ID официанта (положительное число): ");
        waiterId = GetIntInput();
        if (waiterId <= 0)
            Console.WriteLine("ID официанта должно быть положительным числом.");
    } while (waiterId <= 0);

    Console.Write("Введите комментарий (или Enter для пропуска): ");
    var comment = Console.ReadLine() ?? "";

    var order = manager.CreateOrder(tableId, waiterId, comment);

    Console.WriteLine($"Заказ создан успешно! Назначен ID: {order.OrderId}");

    Console.WriteLine("Добавьте блюда в заказ:");
    Console.WriteLine("Доступные блюда:");
    manager.PrintMenu();

    bool addingDishes = true;
    while (addingDishes)
    {
        Console.Write("Введите ID блюда для добавления (или 0 для завершения): ");
        int dishId = GetIntInput();

        if (dishId == 0)
        {
            addingDishes = false;
        }
        else
        {
            var dish = manager.GetDishById(dishId);
            if (dish != null)
            {
                order.AddDish(dish);
                Console.WriteLine($"Блюдо {dish.Name} добавлено в заказ.");
            }
            else
            {
                Console.WriteLine("Блюдо с таким ID не найдено.");
            }
        }
    }

    Console.WriteLine($"Заказ {order.OrderId} полностью сформирован!");
}

```

```

static void EditOrder()
{
    Console.WriteLine("Введите ID заказа для редактирования: ");
    int id = GetIntInput();
    ViewAllOrders();

    var order = manager.GetOrderById(id);
    if (order == null)
    {
        Console.WriteLine("Заказ с таким ID не найден.");
        return;
    }

    order.PrintOrderInfo();

    Console.WriteLine("Что вы хотите изменить?");
    Console.WriteLine("1. ID стола");
    Console.WriteLine("2. Комментарий");
    Console.WriteLine("3. ID официанта");
    Console.WriteLine("4. Всё сразу");
    Console.WriteLine("Выберите: ");

    int choice = GetIntInput();

    switch (choice)
    {
        case 1:
            Console.WriteLine("Введите новый ID стола (положительное число): ");
            int newTableId;
            do
            {
                newTableId = GetIntInput();
                if (newTableId <= 0)
                    Console.WriteLine("ID стола должно быть положительным числом.");
            } while (newTableId <= 0);
            order.ChangeOrder(changeTableId: true, newTableId: newTableId);
            break;
        case 2:
            Console.WriteLine("Введите новый комментарий: ");
            string newComment = Console.ReadLine() ?? "";
            order.ChangeOrder(changeComment: true, newComment: newComment);
            break;
        case 3:
            Console.WriteLine("Введите новый ID официанта (положительное число): ");
            int newWaiterId;
            do
            {
                newWaiterId = GetIntInput();
                if (newWaiterId <= 0)
                    Console.WriteLine("ID официанта должно быть положительным числом.");
            } while (newWaiterId <= 0);
            order.ChangeOrder(changeWaiterId: true, newWaiterId: newWaiterId);
            break;
        case 4:
            Console.WriteLine("Введите новый ID стола (положительное число): ");
            int allTableId;
            do
            {
                allTableId = GetIntInput();
                if (allTableId <= 0)
                    Console.WriteLine("ID стола должно быть положительным числом.");
            } while (allTableId <= 0);

            Console.WriteLine("Введите новый комментарий: ");
            string allComment = Console.ReadLine() ?? "";

            Console.WriteLine("Введите новый ID официанта (положительное число): ");
            int allWaiterId;
            do
            {
                allWaiterId = GetIntInput();
                if (allWaiterId <= 0)
                    Console.WriteLine("ID официанта должно быть положительным числом.");
            } while (allWaiterId <= 0);

            order.ChangeOrder(
                changeTableId: true, newTableId: allTableId,
                changeComment: true, newComment: allComment,

```

```

        changeWaiterId: true, newWaiterId: allWaiterId
    );
    break;
default:
    Console.WriteLine("Неверный выбор.");
    return;
}

Console.WriteLine("Заказ успешно отредактирован!");
}

static void CloseOrder()
{
    Console.Write("Введите ID заказа для закрытия: ");
    int id = GetIntInput();
    ViewAllOrders();

    var order = manager.GetOrderById(id);
    if (order == null)
    {
        Console.WriteLine("Заказ с таким ID не найден.");
        return;
    }

    if (order.IsClosed)
    {
        Console.WriteLine("Заказ уже закрыт.");
        return;
    }

    order.PrintOrderInfo();

    Console.Write("Вы уверены, что хотите закрыть этот заказ? (y/n): ");
    var confirm = Console.ReadLine() ?? "";

    if (confirm.Equals("y", StringComparison.OrdinalIgnoreCase) || confirm.Equals("yes", StringComparison.OrdinalIgnoreCase))
    {
        order.CloseOrder();
        Console.WriteLine("Заказ закрыт успешно!");
    }
    else
    {
        Console.WriteLine("Закрытие заказа отменено.");
    }
}

static void ViewOrder()
{
    Console.Write("Введите ID заказа для просмотра: ");
    int id = GetIntInput();
    ViewAllOrders();

    var order = manager.GetOrderById(id);
    if (order == null)
    {
        Console.WriteLine("Заказ с таким ID не найден.");
        return;
    }

    order.PrintOrderInfo();
}

static void PrintOrderCheck()
{
    Console.Write("Введите ID заказа для печати чека: ");
    int id = GetIntInput();
    ViewAllOrders();

    var order = manager.GetOrderById(id);
    if (order == null)
    {
        Console.WriteLine("Заказ с таким ID не найден.");
        return;
    }

    order.PrintCheck();
}

static void ViewTotalRevenue()

```

```
{
    double revenue = manager.CalculateTotalRevenue();
    Console.WriteLine($"Общая выручка: {revenue:F2} руб.");
}

static void ViewWaiterStats()
{
    Console.Write("Введите ID официанта: ");
    int waiterId = GetIntInput();

    int count = manager.GetWaiterClosedOrdersCount(waiterId);
    Console.WriteLine($"Количество закрытых заказов официанта {waiterId}: {count}");
}

static void PrintDishStatistics()
{
    manager.PrintDishStatistics();
}
}
```

Ответ:

```
C:\Users\Vova\Desktop\Универ X + v
2. Редактировать блюдо по ID
3. Удалить блюдо по ID
4. Просмотреть блюдо по ID

=== ЗАКАЗЫ ===
5. Создать заказ
6. Изменить заказ по ID
7. Закрыть заказ по ID
8. Просмотреть заказ по ID
9. Распечатать чек заказа по ID

=== ОБЩЕЕ ===
10. Просмотреть меню
11. Общая выручка
12. Статистика официанта
13. Статистика по блюдам
14. Просмотреть все заказы
15. Просмотреть все блюда
0. Выход

10
ID: 1
Название: Борщ
Состав: Свекла, капуста, картофель, мясо, вода
Вес: 300/50/30
Цена: 150,00 руб.
Категория: Soup
Время готовки: 20 мин.
Типы: горячее
-----
ID: 2
Название: Оливье
Состав: Картофель, морковь, огурцы, колбаса
Вес: 200/30
Цена: 120,00 руб.
Категория: Salad
Время готовки: 10 мин.
Типы: холодное
-----
ID: 3
Название: Компот
Состав: Ягоды, вода
Вес: 250
Цена: 80,00 руб.
Категория: Beverage
Время готовки: 5 мин.
Типы: веганское
-----

Нажмите любую клавишу для продолжения...
|
```


